Experiment No: 4

Implement the program on Hash functions

Name: Rutuja Dundesh Mane **Roll No: A44**
Div: A **PRN: 2324000718**

Server.java:

```
import java.awt.*;
import java.io.*;
import java.net.*;
import java.security.*;
import java.security.spec.X509EncodedKeySpec;
import java.util.*;
import javax.crypto.*;
import javax.crypto.spec.SecretKeySpec;
import javax.swing.*;
import java.util.Base64;

public class Server extends JFrame {

   private ServerSocket serverSocket;
   private JTextArea logArea;
   private Map<String, String> qpDatabase;

   public Server() {
      setTitle("Exam Server - Diffie Hellman + SHA-256");
      setSize(800, 600);
      setDefaultCloseOperation(EXIT_ON_CLOSE);
      setLayout(new BorderLayout());

      logArea = new JTextArea();
      logArea.setEditable(false);
      add(new JScrollPane(logArea), BorderLayout.CENTER);

      JButton startBtn = new JButton("Start Server (8080)");
      startBtn.addActionListener(e -> startServer());
      add(startBtn, BorderLayout.SOUTH);

      qpDatabase = new HashMap<>();
      qpDatabase.put("10-CS101-1",
         "Q1. What is Java?\nQ2. Explain OOP concepts.\nQ3. Write features of Java.");

      setVisible(true);
   }

   private void startServer() {
```

```
        new Thread(() -> {
            try {
                serverSocket = new ServerSocket(8080);
                log("Server started on port 8080");

                while (true) {
                    Socket client = serverSocket.accept();
                    //log("Client connected: " + client.getInetAddress());
                      log("Client connected " );
                    handleClient(client);
                }
            } catch (Exception e) {
                log("Server Error: " + e.getMessage());
            }
        }).start();
    }

    private void handleClient(Socket client) {
        new Thread(() -> {
            try {
                BufferedReader in = new BufferedReader(new
InputStreamReader(client.getInputStream()));
                PrintWriter out = new PrintWriter(client.getOutputStream(), true);

                String cls = in.readLine();
                String courseCode = in.readLine();
                in.readLine();
                String sem = in.readLine();

                String keyDB = cls + "-" + courseCode + "-" + sem;
                String qp = qpDatabase.get(keyDB);

                if (qp == null) {
                    out.println("ERROR");
                    out.println("Question Paper not found");
                    return;
                }

                MessageDigest sha256 = MessageDigest.getInstance("SHA-256");
                byte[] qpBytes = qp.getBytes("UTF-8");
                byte[] originalHash = sha256.digest(qpBytes);
                String originalHashB64 = Base64.getEncoder().encodeToString(originalHash);
                log("Original QP Hash: " + originalHashB64);

                KeyPairGenerator keyPairGen = KeyPairGenerator.getInstance("DiffieHellman");
                keyPairGen.initialize(2048);
                KeyPair serverKP = keyPairGen.generateKeyPair();

                PublicKey serverPubKey = serverKP.getPublic();
                PrivateKey serverPrivKey = serverKP.getPrivate();
```

```
            log("Server Public Key: " +
Base64.getEncoder().encodeToString(serverPubKey.getEncoded()));

            out.println(Base64.getEncoder().encodeToString(serverPubKey.getEncoded()));

            String clientPubB64 = in.readLine();
            byte[] clientPubBytes = Base64.getDecoder().decode(clientPubB64);
            KeyFactory keyFactory = KeyFactory.getInstance("DiffieHellman");
            PublicKey clientPubKey = keyFactory.generatePublic(new
X509EncodedKeySpec(clientPubBytes));

            KeyAgreement keyAgree = KeyAgreement.getInstance("DiffieHellman");
            keyAgree.init(serverPrivKey);
            keyAgree.doPhase(clientPubKey, true);
            byte[] sharedSecret = keyAgree.generateSecret();

            log("Shared Secret: " + Base64.getEncoder().encodeToString(sharedSecret));

            SecretKey aesKey = new SecretKeySpec(sharedSecret, 0, 16, "AES");
            Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
            cipher.init(Cipher.ENCRYPT_MODE, aesKey);
            byte[] encryptedQP = cipher.doFinal(qpBytes);

            out.println("OK");
            out.println(Base64.getEncoder().encodeToString(encryptedQP));
            out.println(originalHashB64);

            log("QP Sent: Encrypted + Hash");
            client.close();
          } catch (Exception e) {
            log("Client Error: " + e.getMessage());
          }
        }).start();
    }

    private void log(String msg) {
        logArea.append(msg + "\n");
        logArea.setCaretPosition(logArea.getDocument().getLength());
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Server());
    }
}
```

Client.java:

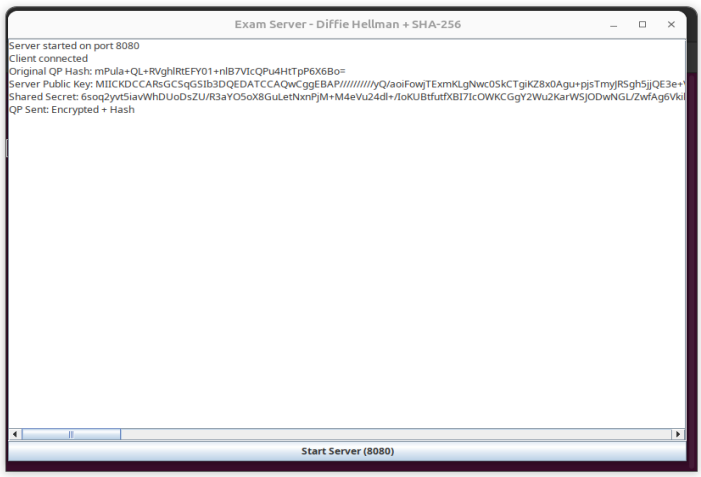
```
import java.awt.*;
import java.awt.event.*;
import java.io.*;
```

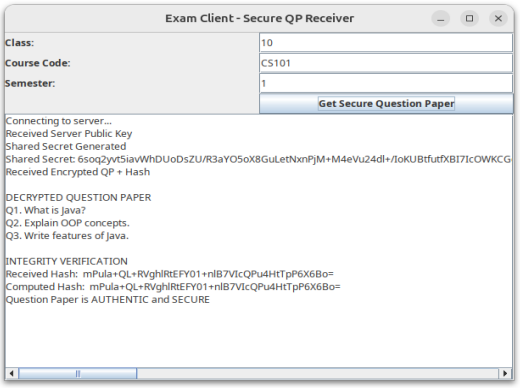
```
import java.net.*;
import java.security.*;
import java.security.spec.X509EncodedKeySpec;
import java.util.Arrays;
import javax.crypto.*;
import javax.crypto.spec.SecretKeySpec;
import javax.swing.*;
import java.util.Base64;

public class Client extends JFrame {

    private JTextField classField, courseField, semField;
    private JTextArea displayArea;
    private JButton connectBtn;
    private Socket socket;

    public Client() {
        setTitle("Exam Client - Secure QP Receiver");
        setSize(600, 500);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLayout(new BorderLayout());

        JPanel inputPanel = new JPanel(new GridLayout(4, 2));
        inputPanel.add(new JLabel("Class:"));
        classField = new JTextField("10");
        inputPanel.add(classField);

        inputPanel.add(new JLabel("Course Code:"));
        courseField = new JTextField("CS101");
        inputPanel.add(courseField);

        inputPanel.add(new JLabel("Semester:"));
        semField = new JTextField("1");
        inputPanel.add(semField);

        connectBtn = new JButton("Get Secure Question Paper");
        connectBtn.addActionListener(e -> requestQuestionPaper());
        inputPanel.add(new JLabel(""));
        inputPanel.add(connectBtn);

        add(inputPanel, BorderLayout.NORTH);

        displayArea = new JTextArea();
        displayArea.setEditable(false);
        add(new JScrollPane(displayArea), BorderLayout.CENTER);

        setVisible(true);
    }

    private void requestQuestionPaper() {
        new Thread(() -> {
```

```
        try {
            displayArea.append("Connecting to server...\n");
            socket = new Socket("localhost", 8080);

            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));

            String cls = classField.getText();
            String course = courseField.getText();
            String sem = semField.getText();

            out.println(cls);
            out.println(course);
            out.println("");
            out.println(sem);



            String serverPubB64 = in.readLine();
            displayArea.append("Received Server Public Key\n");

            KeyPairGenerator keyPairGen = KeyPairGenerator.getInstance("DiffieHellman");
            keyPairGen.initialize(2048);
            KeyPair clientKP = keyPairGen.generateKeyPair();

            PublicKey clientPubKey = clientKP.getPublic();
            PrivateKey clientPrivKey = clientKP.getPrivate();



            out.println(Base64.getEncoder().encodeToString(clientPubKey.getEncoded()));


            KeyFactory keyFactory = KeyFactory.getInstance("DiffieHellman");
            PublicKey serverPubKey = keyFactory.generatePublic(new X509EncodedKeySpec(
                Base64.getDecoder().decode(serverPubB64)));

            KeyAgreement keyAgree = KeyAgreement.getInstance("DiffieHellman");
            keyAgree.init(clientPrivKey);
            keyAgree.doPhase(serverPubKey, true);
            byte[] sharedSecret = keyAgree.generateSecret();

            displayArea.append("Shared Secret Generated\n");
            displayArea.append("Shared Secret: " + Base64.getEncoder().encodeToString(sharedSecret)
+ "\n");

            String status = in.readLine();
            if (!"OK".equals(status)) {
                displayArea.append("Error: " + in.readLine() + "\n");
                socket.close();
```

```
            return;
          }

          String encryptedQPB64 = in.readLine();
          String receivedHashB64 = in.readLine();

          byte[] encryptedQP = Base64.getDecoder().decode(encryptedQPB64);
          byte[] receivedHash = Base64.getDecoder().decode(receivedHashB64);

          displayArea.append("Received Encrypted QP + Hash\n");

          SecretKey aesKey = new SecretKeySpec(sharedSecret, 0, 16, "AES");
          Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
          cipher.init(Cipher.DECRYPT_MODE, aesKey);
          byte[] decryptedQP = cipher.doFinal(encryptedQP);

          String questionPaper = new String(decryptedQP, "UTF-8");
          displayArea.append("\nDECRYPTED QUESTION PAPER\n");
          displayArea.append(questionPaper + "\n");

          MessageDigest sha256 = MessageDigest.getInstance("SHA-256");
          byte[] computedHash = sha256.digest(decryptedQP);

          displayArea.append("\nINTEGRITY VERIFICATION\n");
          displayArea.append("Received Hash:  " +
Base64.getEncoder().encodeToString(receivedHash) + "\n");
          displayArea.append("Computed Hash:  " +
Base64.getEncoder().encodeToString(computedHash) + "\n");

          if (Arrays.equals(receivedHash, computedHash)) {

            displayArea.append("Question Paper is AUTHENTIC and SECURE\n\n");
          } else {
            displayArea.append("TAMPER DETECTED! Do not trust this paper!\n");
          }

          socket.close();


        } catch (Exception e) {
          displayArea.append("Error: " + e.getMessage() + "\n");
        }
      }).start();
    }

    public static void main(String[] args) {
      SwingUtilities.invokeLater(() -> new Client());
    }
}
```

Output:


Exam Server - Diffie Hellman + SHA-256
Server started on port 8080
Client connected
Original QP Hash: mPula+QL+RVghlRtEFY01+nlB7VIcQPu4HtTpP6X6Bo=
Server Public Key: MIICKDCCARsGCSqGSIb3DQEDATCCAQwCggEBAP//////////yQ/aoiFowjTExmKLgNwc0SkCTgiKZ8x0Agu+pjsTmyJRSgh5jjQE3e+
Shared Secret: 6soq2yvt5iavWhDUoDsZU/R3aYO5oX8GuLetNxnPjM+M4eVu24dl+/IoKUBtfutfXBI7IcOWKCGgY2Wu2KarWSJODwNGL/ZwfAg6Vki
QP Sent: Encrypted + Hash
Start Server (8080)


Exam Client - Secure QP Receiver
Class:
10
Course Code:
CS101
Semester:
1
Get Secure Question Paper
Connecting to server...
Received Server Public Key
Shared Secret Generated
Shared Secret: 6soq2yvt5iavWhDUoDsZU/R3aYO5oX8GuLetNxnPjM+M4eVu24dl+/IoKUBtfutfXBI7IcOWKCG
Received Encrypted QP + Hash
DECRYPTED QUESTION PAPER
Q1. What is Java?
Q2. Explain OOP concepts.
Q3. Write features of Java.
INTEGRITY VERIFICATION
Received Hash: mPula+QL+RVghlRtEFY01+nlB7VIcQPu4HtTpP6X6Bo=
Computed Hash: mPula+QL+RVghlRtEFY01+nlB7VIcQPu4HtTpP6X6Bo=
Question Paper is AUTHENTIC and SECURE